

FC7xxx CRC User Manual

Rev.0.6

Revision History

Revision	Date	Changes
0.1	2023/12/15	Initial release
0.6	2023/03/29	Updated for MCAL V0.6.0 - Added support for FC7240

Table of Contents

Chapter 1 CRC Introduction	5
1.1 Requirement Tracing.....	5
1.2 Hardware Summary.....	5
Chapter 2 Software Design	6
2.1 Rejected Requirements	6
2.2 File Structure.....	6
2.3 Macros	8
2.3.1 Macros in Crc.h.....	8
2.3.2 Macros in Crc_Version.h	9
2.3.3 Macros in Crc_Cfg.h	9
2.4 Enums.....	12
2.4.1 Enumerations in Crc_Types.h	12
2.4.2 Enumerations in Crc_RegOps.h	13
2.5 Structures.....	14
2.5.1 Crc_InitType.....	14
2.5.2 CRC_ChannelConfiguration	14
2.5.3 Crc_ConfigType.....	14
2.6 API Functions	15
2.6.1 Functions in Crc.h	15
2.7 Hardware Functions	20
2.7.1 Functions in Crc_Hw.h.....	20
2.8 Software Functions.....	22
2.8.1 Functions in Crc_Lookup_Tables.h	22
2.8.2 Functions in Crc_RunTime.h.....	24
2.9 API Sequence Diagram	25
2.9.1 Crc_CalculateCRC8().....	25
2.9.2 Crc_CalculateCRC8H2F()	25
2.9.3 Crc_CalculateCRC16().....	26
2.9.4 Crc_CalculateCRC16ARC().....	26
2.9.5 Crc_CalculateCRC32().....	27
2.9.6 Crc_CalculateCRC32P4()	27
2.9.7 Crc_CalculateCRC64().....	28
Chapter 3 Tresos Configuration Items	29
3.1 Container Inclusion Relation.....	29
3.1.1 IMPLEMENTATION_CONFIG_VARIANT.....	29

3.1.2	CrcGeneral.....	30
3.1.3	CrcChannelConfig.....	30
3.1.4	CommonPublishedInformation.....	31
3.2	Containers and Variables	31
3.3	IMPLEMENTATION_CONFIG_VARIANT.....	31
3.3.1	CrcGeneral.....	31
3.3.2	CrcChannelConfig.....	36
3.3.3	CrcEcucPartitionRef.....	38
3.3.4	CommonPublishedInformation.....	38
Chapter 4 Configuration Guides.....		41
4.1	Configuration Item Constraint.....	41
4.2	CRC Usage Common Steps.....	41

Chapter 1 CRC Introduction

1.1 Requirement Tracing

The design of this module follows the specifications of the CRC driver specified in AUTOSAR Classic Platform Release 4.6.0. For detailed requirements, refer to the *AUTOSAR_SWS_CRCLibrary*.

1.2 Hardware Summary

The FC7300F8MDT and FC7300F4MDD support two CRC instances, and the FC7300F4MDS and FC7240F2MDS support one CRC instance. The CRC hardware module includes the following features:

- Programmable initial seed value and polynomial
- Support the bit/byte swap operation on input/output data
- Option for complement of final CRC result
- 32-bit CPU register programming interface
- Hardware CRC generator circuit using a 8-bit(only supported in FC7240F2MDS), 16-bit or 32-bit programmable shift register

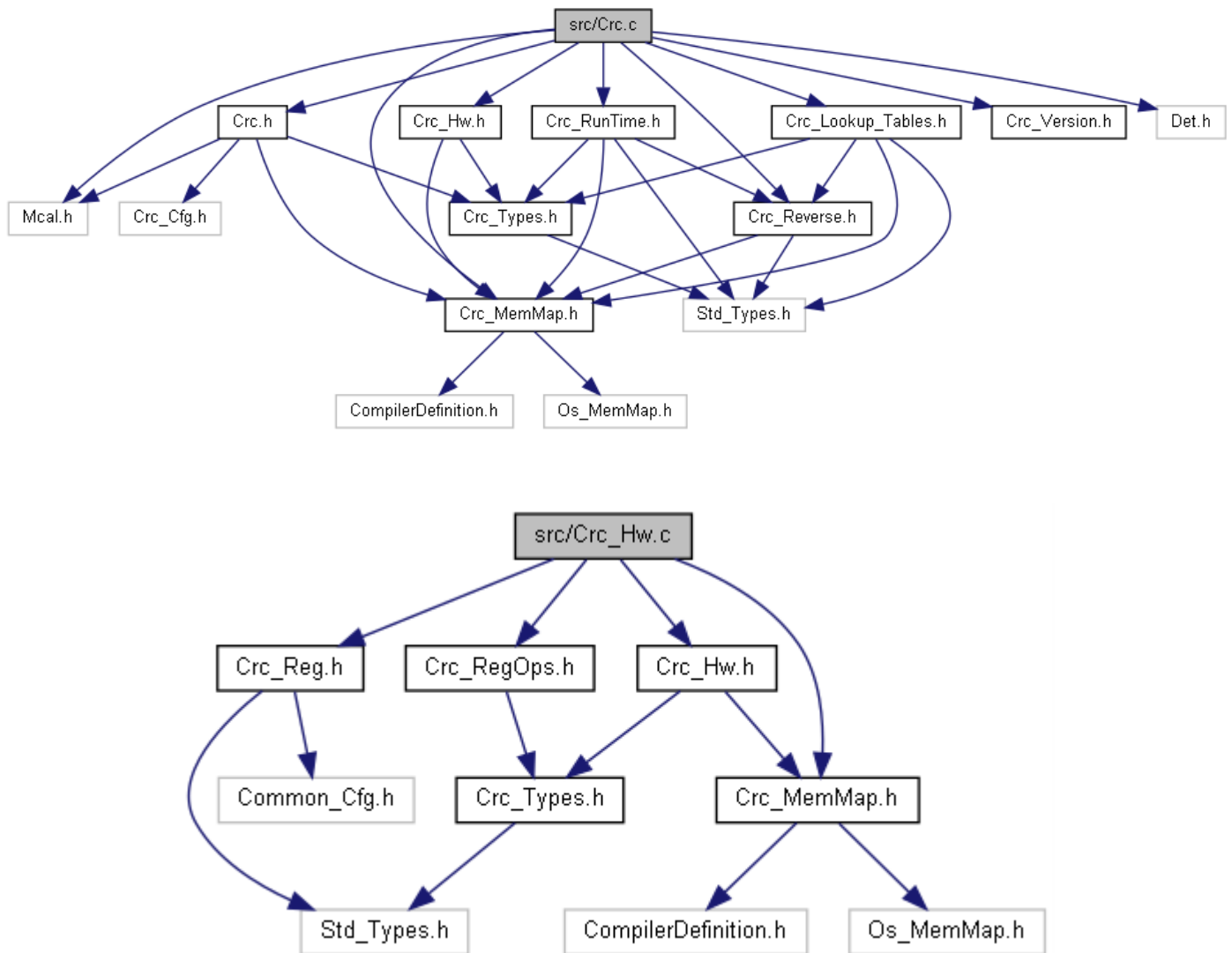
Chapter 2 Software Design

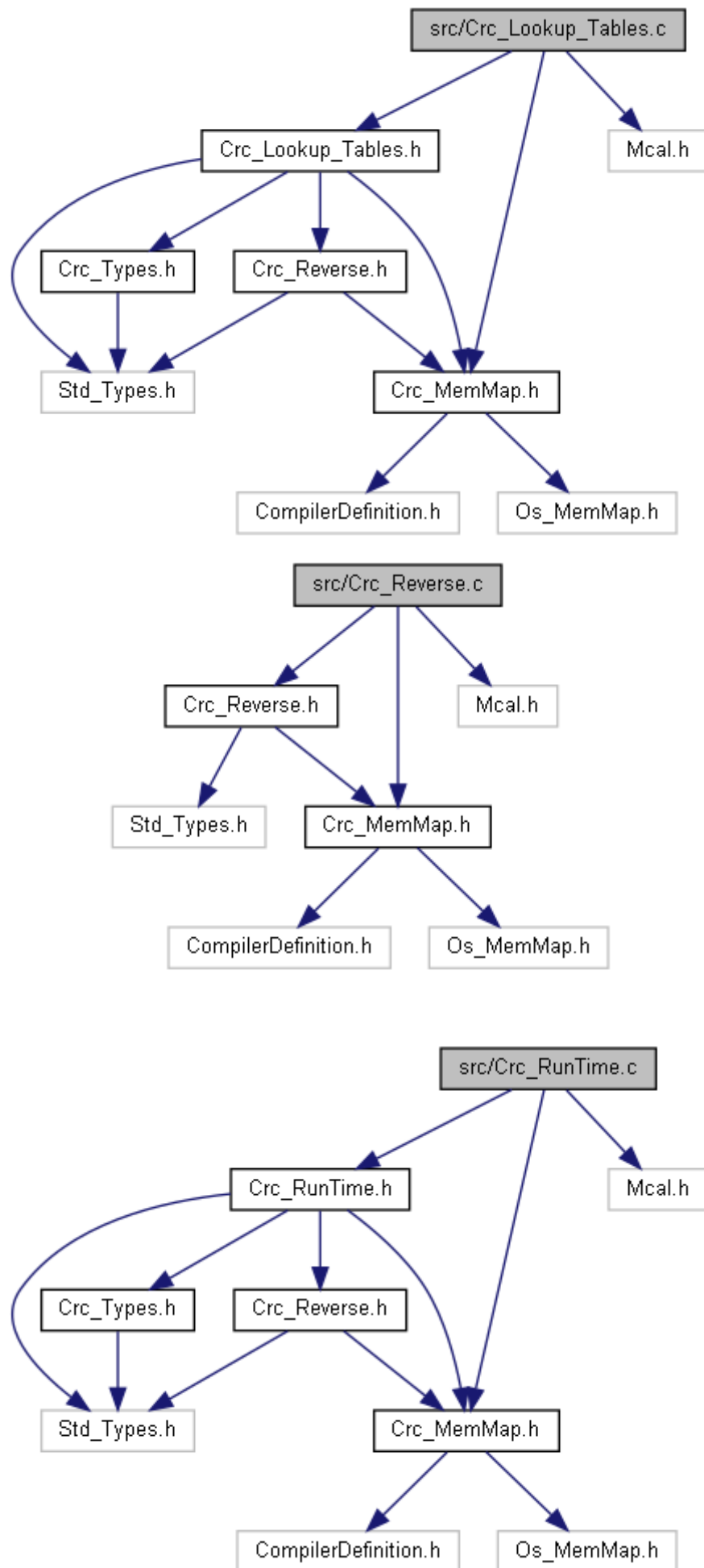
2.1 Rejected Requirements

Rejected Requirement 4 **SWS_CRC_00065**

Description	If the CRC calculation within the function Crc_CalculateCRC64 is performed by hardware, then the CRC module's implementer shall ensure reentrancy of this function by implementing a (software based) locking mechanism.
Rejection Reason	There are no platforms supporting Crc64 Hardware calculation (Hardware limitation).

2.2 File Structure





2.3 Macros

2.3.1 Macros in Crc.h

- **#define CRC_CALCULATECRC8_ID** ((uint8)0x01U)
API service ID for Crc_CalculateCRC8 function
- **#define CRC_CALCULATECRC16_ID** ((uint8)0x02U)
API service ID for Crc_CalculateCRC16 function
- **#define CRC_CALCULATECRC32_ID** ((uint8)0x03U)
API service ID for Crc_CalculateCRC32 function.
- **#define CRC_GETVERSIONINFO_ID** ((uint8)0x04U)
API service ID for Crc_GetVersionInfo function.
- **#define CRC_CALCULATECRC8H2F_ID** ((uint8)0x05U)
API service ID for Crc_CalculateCRC8H2F function.
- **#define CRC_CALCULATECRC32P4_ID** ((uint8)0x06U)
API service ID for Crc_CalculateCRC32P4 function.
- **#define CRC_CALCULATECRC64_ID** ((uint8)0x07U)
API service ID for Crc_CalculateCRC64 function.
- **#define CRC_CALCULATECRC16ARC_ID** ((uint8)0x08U)
API service ID for Crc_CalculateCRC16ARC function.
- **#define CRC_CALCULATECHANNEL_ID** ((uint8)0x0AU)
API service ID for Crc_SetChannelCalculate function.
- **#define CRC_E_INVALID_CHANNEL** ((uint8)0x0BU)
API service is called with wrong channel identifier.
- **#define CRC_E_INVALID_POINTER** ((uint8)0x0CU)
API service is called with NULL pointer parameter.
- **#define CRC_E_INIT_PARTITION** ((uint8)0x0EU)
The CRC module calls the incorrect core

2.3.2 Macros in Crc_Version.h

- `#define CRC_AR_RELEASE_MAJOR_VERSION 4`
- `#define CRC_AR_RELEASE_MINOR_VERSION 6`
- `#define CRC_AR_RELEASE_REVISION_VERSION 0`
- `#define CRC_MODULE_ID 201`
- `#define CRC_SW_MAJOR_VERSION 0`
- `#define CRC_SW_MINOR_VERSION 6`
- `#define CRC_SW_PATCH_VERSION 0`
- `#define CRC_VENDOR_ID 174`

2.3.3 Macros in Crc_Cfg.h

- `#define CRC_CFG_AR_RELEASE_MAJOR_VERSION 4`
- `#define CRC_CFG_AR_RELEASE_MINOR_VERSION 6`
- `#define CRC_CFG_AR_RELEASE_REVISION_VERSION 0`
- `#define CRC_CFG_SW_MAJOR_VERSION 0`
- `#define CRC_CFG_SW_MINOR_VERSION 6`
- `#define CRC_CFG_SW_PATCH_VERSION 0`
- `#define CRC_CFG_VENDOR_ID 174`
- `#define CRC_CFG_MODULE_ID 201`
- `#define CRC_CALCULATECHANNEL (STD_ON)`
Switch for enabling the Crc_SetChannelCalculate as autosar library.
- `#define CRC_CRC8_USED (STD_ON)`
Switch for enabling the Crc_CalculateCRC8 as autosar library.
- `#define CRC_CRC8H2F_USED (STD_ON)`

Switch for enabling the Crc_CalculateCRC8H2F as autosar library.

- #define **CRC_CRC16_USED** (STD_ON)

Switch for enabling the Crc_CalculateCRC16 as autosar library.

- #define **CRC_CRC16ARC_USED** (STD_ON)

Switch for enabling the Crc_CalculateCRC16ARC as autosar library.

- #define **CRC_CRC32_USED** (STD_ON)

Switch for enabling the Crc_CalculateCRC32 as autosar library.

- #define **CRC_CRC32P4_USED** (STD_ON)

Switch for enabling the Crc_CalculateCRC32P4 as autosar library.

- #define **CRC_CRC64_USED** (STD_ON)

Switch for enabling the Crc_CalculateCRC64 as autosar library.

- #define **CRC_MUTILCORE_SUPPORT** (STD_OFF)

Switch for enabling the multicore support.

- #define **CRC_DEV_ERROR_DETECT** (STD_ON)

Switch for enabling the development error detection.

- #define **CRC_GET_VERSION_INFO_API** (STD_ON)

Switch to indicate that Crc_GetVersionInfo API is supported.

- #define **CRC_HW_INSTANCE_0** ((uint8)0U)

CRC instance 0.

- #define **CRC_HW_INSTANCE_1** ((uint8)1U)

CRC instance 1.

- #define **CRC_INSTANCE_COUNT_U8** ((uint8)2U)

Number of CRC instances supported by the platform.

- #define **CRC_8BIT_SAE_J1850** (CRC_TABLE_CALCULATION)
Crc8Mode calculation type.
- #define **CRC_8BIT_H2F** (CRC_TABLE_CALCULATION)
Crc8H2FMode calculation type.
- #define **CRC_16BIT_CCITT_FALSE** (CRC_TABLE_CALCULATION)
Crc16Mode calculation type.
- #define **CRC_16BIT_ARC** (CRC_TABLE_CALCULATION)
Crc16ARCMODE calculation type.
- #define **CRC_32BIT_ETHERNET** (CRC_TABLE_CALCULATION)
Crc32Mode calculation type.
- #define **CRC_32BIT_E2E_P4** (CRC_TABLE_CALCULATION)
Crc32P4Mode calculation type.
- #define **CRC_64BIT_ECMA** (CRC_TABLE_CALCULATION)
Crc64Mode calculation type.
- #define **Crc16InstanceCoreID** (3)
When Crc16Mode uses hardware calculation type, if the multicore is enabled, the value is the core which distribute to the hardware, if the multicore is disabled, the value is 3.
- #define **Crc16ARCInstanceCoreID** (3)
When Crc16ARCMODE uses hardware calculation type, if the multicore is enabled, the value is the core which distribute to the hardware, if the multicore is disabled, the value is 3.
- #define **Crc32InstanceCoreID** (3)
When Crc32Mode uses hardware calculation type, if the multicore is enabled, the value is the core which distribute to the hardware, if the multicore is disabled, the value is 3.
- #define **Crc32P4InstanceCoreID** (3)
When Crc32P4Mode uses hardware calculation type, if the multicore is enabled, the value is the core which distribute to the hardware, if the multicore is disabled, the value is 3.

- **#define `Crc8Instance`** (2)
When `Crc8Mode` uses hardware calculation type, the value is the instance number, otherwise the value is 2.
- **#define `Crc8H2FInstance`** (2)
When `Crc8H2FMode` uses hardware calculation type, the value is the instance number, otherwise the value is 2
- **#define `Crc16Instance`** (2)
When `Crc16Mode` uses hardware calculation type, the value is the instance number, otherwise the value is 2.
- **#define `Crc16ARCInstance`** (2)
When `Crc16ARCMODE` uses hardware calculation type, the value is the instance number, otherwise the value is 2
- **#define `Crc32Instance`** (2)
When `Crc32Mode` uses hardware calculation type, the value is the instance number, otherwise the value is 2
- **#define `Crc32P4Instance`** (2)
When `Crc32P4Mode` uses hardware calculation type, the value is the instance number, otherwise the value is 2
- **#define `CRC_NUM_LOGIC_CHANNEL_MAX`** ((uint32)1U)
The number of channels configured in the `CrcChannelConfig` Tab.
- **#define `CrcConf_CrcChannelConfig_CrcChannelConfig_0`** ((uint32)0U)
The index of channels configured in the `CrcChannelConfig` Tab.

2.4 Enums

2.4.1 Enumerations in `Crc_Types.h`

2.4.1.1 `CRC_BitWidthType`

Enumeration	<code>CRC_BitWidthType</code>	
Description	This type contains the CRC Width Type.	
Values	Value	Description
	<code>CRC_WIDTH_8BIT = 0U</code>	The CRC width is 8 bit.
	<code>CRC_WIDTH_16BIT = 1U</code>	The CRC width is 16 bit.
	<code>CRC_WIDTH_32BIT = 2U</code>	The CRC width is 32 bit.
	<code>CRC_WIDTH_64BIT = 3U</code>	The CRC width is 64 bit.
	<code>CRC_BIT_INVALID = 4</code>	The CRC width is not supported on this platform.

2.4.1.2 `Crc_StandardType`

Enumeration	<code>Crc_StandardType</code>
-------------	-------------------------------

Description	The supported CRC protocol type.	
Values	Value	Description
	CRC_PROTOCOL_8BIT_H2F = 0	8-bit CRC H2F code with Polynomial: 0x2F
	CRC_PROTOCOL_8BIT_SAE_J1850 = 1	8-bit CRC SAE-J1850 code with Polynomial: 0x1D
	CRC_PROTOCOL_16BIT_ARC = 2	16-bit CRC ARC code with Polynomial: 0x8005
	CRC_PROTOCOL_16BIT_CCITT_FALSE = 3	16-bit CRC CCITT16 code with Polynomial: 0x1021
	CRC_PROTOCOL_32BIT_E2E_P4 = 4	32-bit CRC E2E Profile 4 code with Polynomial: 0xF4ACFB13
	CRC_PROTOCOL_32BIT_ETHERNET = 5	32-bit CRC Ethernet CCITT32 code with Polynomial: 0x04C11DB7
	CRC_PROTOCOL_64BIT_ECMA = 6	64-bit CRC: ECMA

2.4.1.3 CRC_WriteDataSwapType

Enumeration	CRC_WriteDataSwapType	
Description	The data swap for write.	
Values	Value	Description
	WRITE_DATASWAP_NONE = 0	none data swap for write
	WRITE_DATASWAP_BIT = 1	only bits data swap for write
	WRITE_DATASWAP_BIT_BYTE = 2	both bits and bytes data swap for write
	WRITE_DATASWAP_BYTE = 3	only bytes data swap for write

2.4.1.4 CRC_ReadDataSwapType

Enumeration	CRC_ReadDataSwapType	
Description	The data swap for read.	
Values	Value	Description
	READ_DATASWAP_NONE = 0	none data swap for read
	READ_DATASWAP_BIT = 1	only bits data swap for read
	READ_DATASWAP_BIT_BYTE = 2	both bits and bytes data swap for read
	READ_DATASWAP_BYTE = 3	only bytes data swap for read

2.4.1.5 CRC_ReadDataFXORType

Enumeration	CRC_ReadDataFXORType	
Description	The data swap for read.	
Values	Value	Description
	READ_DATA_NORMAL = 0	none complement of reading CRC data
	READ_DATA_FXOR = 1	Invert or complement with 0xFFFFFFFF or 0xFFFF of crc data

2.4.2 Enumerations in Crc_RegOps.h

2.4.2.1 CRC_WriteCommandType

Enumeration	CRC_WriteCommandType	
Description	The command type of write crc data or seed value	
Values	Value	Description

	WRITE_COMMAND_DATA = 0	FTU counter stopped, channel works as function mode.
	WRITE_COMMAND_SEED = 1	FTU counter stopped, channel output force to safe state.

2.5 Structures

2.5.1 Crc_InitType

Structure	Crc_InitType
Description	Defines the crc protocol information
Diagram	N/A
Data Fields	- uint64 PolynomValue CRC PolynomValue. - uint64 InitialSeedValue CRC initial seed data. - CRC_WriteDataSwapType eWriteDataSwap The CRC write data bit and byte swap or not. - CRC_ReadDataSwapType eReadDataSwap The CRC read data bit and byte swap or not. - CRC_ReadDataFXORType eReadDataFXOR The CRC result inverse or not. - CRC_BitWidthType eBitWidth The CRC width.

2.5.2 CRC_ChannelConfiguration

Structure	CRC_ChannelConfiguration
Description	Defines the crc channel configuration
Diagram	<pre> graph BT CRC_ChannelConfiguration -- sCrclnitType --> Crc_InitType </pre>
Data Fields	- uint8 u8Instance The CRC hardware instance. If the calculation type chooses table and runtime, this parameter should ignore. - Crc_CalculationType eCalType The CRC protocol. - Crc_ProtocolType eProtocolType The choosen CRC protocol information. - Crc_InitType sCrclnitType The CRC information.

2.5.3 Crc_ConfigType

Structure	Crc_ConfigType
------------------	----------------

Description	Defines the crc structure
Diagram	<pre> graph BT Crc_InitType[Crc_InitType] CRC_ChannelConfiguration[CRC_ChannelConfiguration] Crc_ConfigType[Crc_ConfigType] CRC_ChannelConfiguration -.-> sCrcInitType Crc_InitType Crc_ConfigType -.-> pCrcChannelsConfig CRC_ChannelConfiguration </pre>
Data Fields	- uint8 nNumChannels The CRC channel number. - const CRC_ChannelConfiguration(* pCrcChannelsConfig)[] The channel configuration. - const uint8 * CrcPartitionMappingPtr Multicore module partion map table.

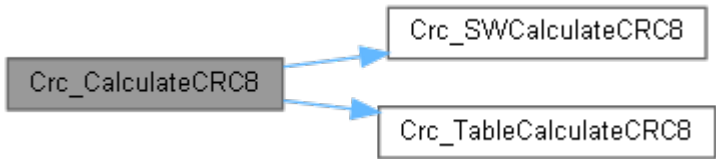
2.6 API Functions

2.6.1 Functions in Crc.h

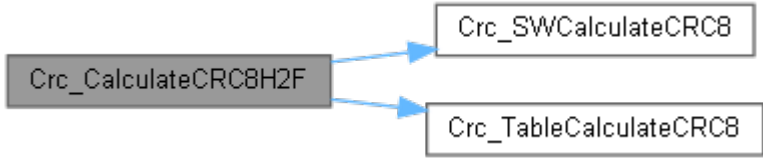
2.6.1.1 uint64 Crc_CalculateChannel (const uint32 Channel, const Crc_ConfigType *ConfigPtr, const uint8 *pCrcData, uint32 CrcLength, uint64 CrcStartValue, boolean IsFirstCall)

Function	uint64 Crc_SetChannelCalculate(const uint32 Channel, const Crc_ConfigType * ConfigPtr, const uint8 * pCrcData, uint32 CrcLength, uint64 CrcStartValue, boolean IsFirstCall);	
Description	This service is a reentrant function that shall start the Channel to calculate the CRC using the configured algorithm.	
Diagram	N/A	
Parameters	Parameter	Description
	Channel	The index of CRC channel for current configuration structure.
	ConfigPtr	Pointer to a selected configuration structure.
	pCrcData	Pointer to the Crc Data Input.
	CrcLength	Length of crcDataPtr block to be calculated in bytes
	CrcStartValue	Start value (seed Value) when the algorithm starts.
	IsFirstCall	- TRUE: ignore CrcStartValue the initialization value is known by the chosen algorithm. - FALSE: initialization CrcStartValue is interpreted to be the return value of the previous function
Returns	CRC calculation results for the selected channel configuration.	

2.6.1.2 `uint8 Crc_CalculateCRC8(const uint8 *Crc_DataPtr, uint32 Crc_Length, uint8 Crc_StartValue8, boolean Crc_IsFirstCall)`

Function	<code>uint8 Crc_CalculateCRC8(const uint8 *Crc_DataPtr, uint32 Crc_Length, uint8 Crc_StartValue8, boolean Crc_IsFirstCall)</code>	
Description	This function calculates CRC8 (SAE J1850) and returns the result.	
Diagram	 <pre> graph LR A[Crc_CalculateCRC8] --> B[Crc_SWCalculateCRC8] A --> C[Crc_TableCalculateCRC8] </pre>	
Parameters	Parameter	Description
	<code>Crc_DataPtr</code>	Pointer to the Crc Data Input.
	<code>Crc_Length</code>	Length of <code>crcDataPtr</code> block to be calculated in bytes
	<code>Crc_StartValue8</code>	Start value (seed Value) when the algorithm starts.
	<code>Crc_IsFirstCall</code>	- TRUE: ignore <code>CrcStartValue</code> the initialization value is known by the chosen algorithm. - FALSE: initialization <code>CrcStartValue</code> is interpreted to be the return value of the previous function
Returns	CRC8 SAE J1850 result	

2.6.1.3 `uint8 Crc_CalculateCRC8H2F(const uint8 *Crc_DataPtr, uint32 Crc_Length, uint8 Crc_StartValue8H2F, boolean Crc_IsFirstCall)`

Function	<code>uint8 Crc_CalculateCRC8H2F(const uint8 *Crc_DataPtr, uint32 Crc_Length, uint8 Crc_StartValue8H2F, boolean Crc_IsFirstCall)</code>	
Description	This function calculates CRC8 (H2F) and returns the result	
Diagram	 <pre> graph LR A[Crc_CalculateCRC8H2F] --> B[Crc_SWCalculateCRC8] A --> C[Crc_TableCalculateCRC8] </pre>	
Parameters	Parameter	Description
	<code>Crc_DataPtr</code>	Pointer to the Crc Data Input.
	<code>Crc_Length</code>	Length of <code>crcDataPtr</code> block to be calculated in bytes
	<code>Crc_StartValue8</code>	Start value (seed Value) when the algorithm starts.
	<code>Crc_IsFirstCall</code>	- TRUE: ignore <code>CrcStartValue</code> the initialization value is known by the chosen algorithm. - FALSE: initialization <code>CrcStartValue</code> is interpreted to be the return value of the previous function
Returns	CRC8 H2F result	

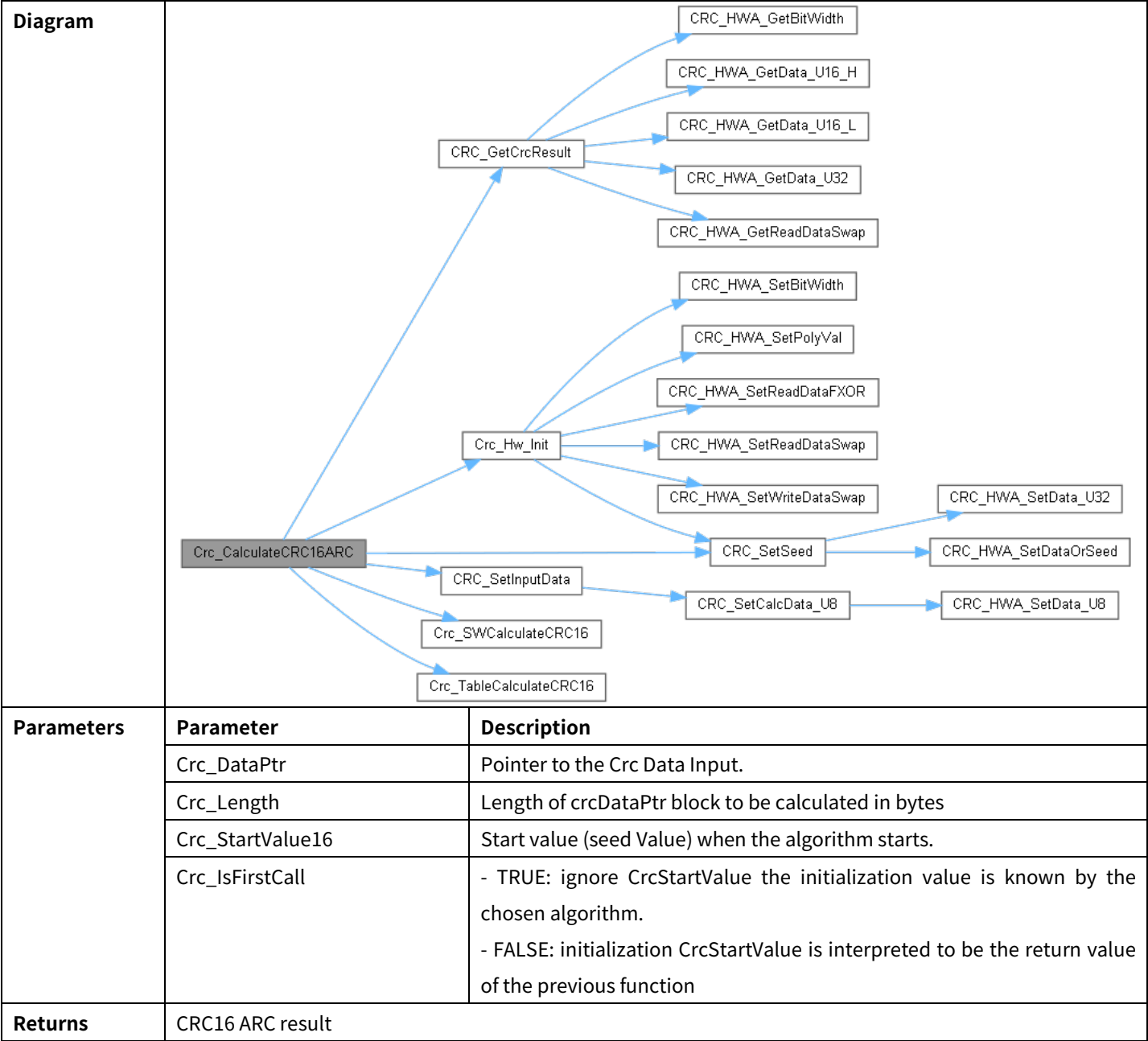
2.6.1.4 `uint16 Crc_CalculateCRC16(const uint8 *Crc_DataPtr, uint32 Crc_Length, uint16 Crc_StartValue16, boolean Crc_IsFirstCall)`

Function	<code>uint16 Crc_CalculateCRC16(const uint8 *Crc_DataPtr, uint32 Crc_Length, uint16 Crc_StartValue16, boolean Crc_IsFirstCall)</code>
-----------------	---

Description	This function calculates CRC16 (CCITT_FALSE) and returns the result	
Diagram	<pre> graph LR Crc_CalculateCRC16 --> Crc_Hw_Init Crc_CalculateCRC16 --> Crc_SWCalculateCRC16 Crc_CalculateCRC16 --> Crc_TableCalculateCRC16 Crc_Hw_Init --> CRC_GetCrcResult Crc_Hw_Init --> CRC_HWA_SetBitWidth Crc_Hw_Init --> CRC_HWA_SetPolyVal Crc_Hw_Init --> CRC_HWA_SetReadDataFXOR Crc_Hw_Init --> CRC_HWA_SetReadDataSwap Crc_Hw_Init --> CRC_HWA_SetWriteDataSwap CRC_GetCrcResult --> CRC_HWA_GetBitWidth CRC_GetCrcResult --> CRC_HWA_GetData_U16_H CRC_GetCrcResult --> CRC_HWA_GetData_U16_L CRC_GetCrcResult --> CRC_HWA_GetData_U32 CRC_GetCrcResult --> CRC_HWA_GetReadDataSwap Crc_SWCalculateCRC16 --> CRC_SetInputData Crc_SWCalculateCRC16 --> CRC_SetCalcData_U8 CRC_SetInputData --> CRC_SetSeed CRC_SetInputData --> CRC_SetCalcData_U8 CRC_SetSeed --> CRC_HWA_SetData_U32 CRC_SetSeed --> CRC_HWA_SetDataOrSeed CRC_SetCalcData_U8 --> CRC_HWA_SetData_U8 </pre>	
Parameters	Parameter	Description
	Crc_DataPtr	Pointer to the Crc Data Input.
	Crc_Length	Length of crcDataPtr block to be calculated in bytes
	Crc_StartValue16	Start value (seed Value) when the algorithm starts.
	Crc_IsFirstCall	- TRUE: ignore CrcStartValue the initialization value is known by the chosen algorithm. - FALSE: initialization CrcStartValue is interpreted to be the return value of the previous function
Returns	CRC16 CCITT FALSE result	

2.6.1.5 `uint16 Crc_CalculateCRC16ARC(const uint8 *Crc_DataPtr, uint32 Crc_Length, uint16 Crc_StartValue16, boolean Crc_IsFirstCall)`

Function	<code>uint16 Crc_CalculateCRC16ARC(const uint8 *Crc_DataPtr, uint32 Crc_Length, uint16 Crc_StartValue16, boolean Crc_IsFirstCall)</code>
Description	This function calculates CRC16 (ARC) and returns the result



2.6.1.6 uint32 Crc_CalculateCRC32(const uint8 *Crc_DataPtr, uint32 Crc_Length, uint32 Crc_StartValue32, boolean Crc_IsFirstCall)

Function	uint32 Crc_CalculateCRC32(const uint8 *Crc_DataPtr, uint32 Crc_Length, uint32 Crc_StartValue32, boolean Crc_IsFirstCall)
Description	CRC32 (Ethernet) result

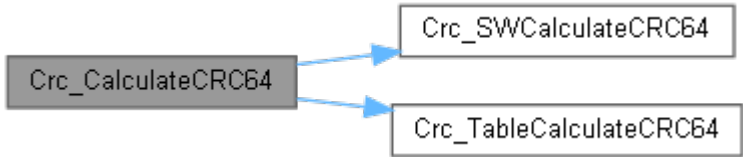
Diagram											
Parameters	<table border="1"> <thead> <tr> <th>Parameter</th><th>Description</th></tr> </thead> <tbody> <tr> <td>Crc_DataPtr</td><td>Pointer to the Crc Data Input.</td></tr> <tr> <td>Crc_Length</td><td>Length of crcDataPtr block to be calculated in bytes</td></tr> <tr> <td>Crc_StartValue32</td><td>Start value (seed Value) when the algorithm starts.</td></tr> <tr> <td>Crc_IsFirstCall</td><td>- TRUE: ignore CrcStartValue the initialization value is known by the chosen algorithm. - FALSE: initialization CrcStartValue is interpreted to be the return value of the previous function</td></tr> </tbody> </table>	Parameter	Description	Crc_DataPtr	Pointer to the Crc Data Input.	Crc_Length	Length of crcDataPtr block to be calculated in bytes	Crc_StartValue32	Start value (seed Value) when the algorithm starts.	Crc_IsFirstCall	- TRUE: ignore CrcStartValue the initialization value is known by the chosen algorithm. - FALSE: initialization CrcStartValue is interpreted to be the return value of the previous function
Parameter	Description										
Crc_DataPtr	Pointer to the Crc Data Input.										
Crc_Length	Length of crcDataPtr block to be calculated in bytes										
Crc_StartValue32	Start value (seed Value) when the algorithm starts.										
Crc_IsFirstCall	- TRUE: ignore CrcStartValue the initialization value is known by the chosen algorithm. - FALSE: initialization CrcStartValue is interpreted to be the return value of the previous function										
Returns	CRC32 (Ethernet) result										

2.6.1.7 `uint32 Crc_CalculateCRC32P4(const uint8 *Crc_DataPtr, uint32 Crc_Length, uint32 Crc_StartValue32, boolean Crc_IsFirstCall)`

Function	uint32 Crc_CalculateCRC32P4(const uint8 *Crc_DataPtr, uint32 Crc_Length, uint32 Crc_StartValue32, boolean Crc_IsFirstCall)										
Description	CRC32 (P4) result										
Diagram											
Parameters	<table border="1"> <thead> <tr> <th>Parameter</th><th>Description</th></tr> </thead> <tbody> <tr> <td>Crc_DataPtr</td><td>Pointer to the Crc Data Input.</td></tr> <tr> <td>Crc_Length</td><td>Length of crcDataPtr block to be calculated in bytes</td></tr> <tr> <td>Crc_StartValue32</td><td>Start value (seed Value) when the algorithm starts.</td></tr> <tr> <td>Crc_IsFirstCall</td><td>- TRUE: ignore CrcStartValue the initialization value is known by the chosen algorithm.</td></tr> </tbody> </table>	Parameter	Description	Crc_DataPtr	Pointer to the Crc Data Input.	Crc_Length	Length of crcDataPtr block to be calculated in bytes	Crc_StartValue32	Start value (seed Value) when the algorithm starts.	Crc_IsFirstCall	- TRUE: ignore CrcStartValue the initialization value is known by the chosen algorithm.
Parameter	Description										
Crc_DataPtr	Pointer to the Crc Data Input.										
Crc_Length	Length of crcDataPtr block to be calculated in bytes										
Crc_StartValue32	Start value (seed Value) when the algorithm starts.										
Crc_IsFirstCall	- TRUE: ignore CrcStartValue the initialization value is known by the chosen algorithm.										

		- FALSE: initialization CrcStartValue is interpreted to be the return value of the previous function
Returns	CRC32 (P4) result	

2.6.1.8 uint64 Crc_CalculateCRC64(const uint8 *Crc_DataPtr, uint32 Crc_Length, uint64 Crc_StartValue64, boolean Crc_IsFirstCall)

Function	uint64 Crc_CalculateCRC64(const uint8 *Crc_DataPtr, uint32 Crc_Length, uint64 Crc_StartValue64, boolean Crc_IsFirstCall)	
Description	CRC64 (ECMA) result	
Diagram	 <pre> graph LR A[Crc_CalculateCRC64] --> B[Crc_SWCalculateCRC64] A --> C[Crc_TableCalculateCRC64] </pre>	
Parameters	Parameter	Description
	Crc_DataPtr	Pointer to the Crc Data Input.
	Crc_Length	Length of crcDataPtr block to be calculated in bytes
	Crc_StartValue64	Start value (seed Value) when the algorithm starts.
	Crc_IsFirstCall	- TRUE: ignore CrcStartValue the initialization value is known by the chosen algorithm. - FALSE: initialization CrcStartValue is interpreted to be the return value of the previous function
Returns	CRC64 (ECMA) result	

2.6.1.9 void Crc_GetVersionInfo(Std_VersionInfoType *versioninfo)

Function	void Crc_GetVersionInfo(Std_VersionInfoType *Versioninfo)	
Description	Return the version information of this module	
Diagram	N/A	
Parameters	Parameter	Description
	Versioninfo	pointer to Std_VersionInfoType output variable.
Returns	N/A	

2.7 Hardware Functions

2.7.1 Functions in Crc_Hw.h

2.7.1.1 void Crc_Hw_Init(const Crc_InitType pInitCfg, const uint8 u8Instance)

Function	void Crc_Hw_Init(const Crc_InitType pInitCfg, const uint8 u8Instance)
Description	Hardware level initializes the CRC Driver settings based on the configuration.

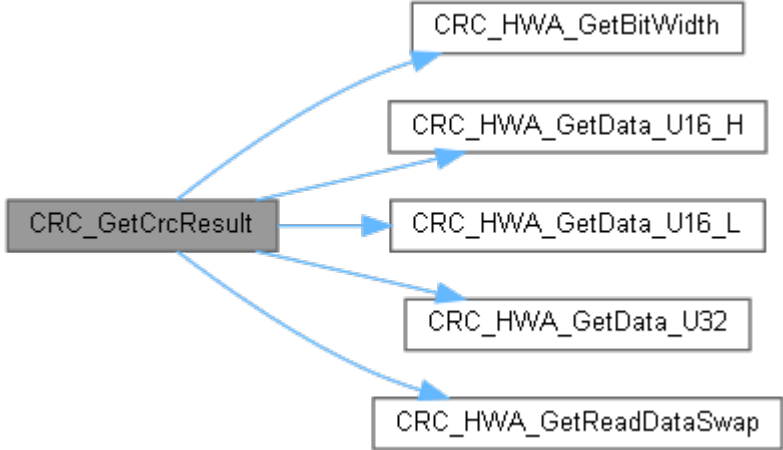
Diagram	<pre> graph LR Crc_Hw_Init[Crc_Hw_Init] --> CRC_HWA_SetBitWidth[CRC_HWA_SetBitWidth] Crc_Hw_Init --> CRC_HWA_SetPolyVal[CRC_HWA_SetPolyVal] Crc_Hw_Init --> CRC_HWA_SetReadDataFXOR[CRC_HWA_SetReadDataFXOR] Crc_Hw_Init --> CRC_HWA_SetReadDataSwap[CRC_HWA_SetReadDataSwap] Crc_Hw_Init --> CRC_HWA_SetWriteDataSwap[CRC_HWA_SetWriteDataSwap] Crc_Hw_Init --> CRC_SetSeed[CRC_SetSeed] CRC_SetSeed --> CRC_HWA_SetData_U32[CRC_HWA_SetData_U32] CRC_SetSeed --> CRC_HWA_SetDataOrSeed[CRC_HWA_SetDataOrSeed] </pre>	
Parameters	Parameter	Description
	pInitCfg	Structure use to initialize the Crc hardware.
	u8Instance	CRC instance index.
Returns	N/A	
Referenced By	Crc_Init()	

2.7.1.2 void CRC_SetInputData(uint8 u8Instance, const uint8 pData[], uint32 u32DataSize)

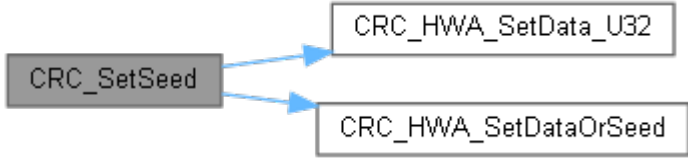
Function	void CRC_SetInputData(uint8 u8Instance, const uint8 pData[], uint32 u32DataSize)	
Description	Hardware level de-initializes the CRC Driver settings.	
Diagram	<pre> graph LR CRC_SetInputData[CRC_SetInputData] --> CRC_HWA_SetData_U8[CRC_HWA_SetData_U8] </pre>	
Parameters	Parameter	Description
	u8Instance	CRC instance index.
	pData	CRC input data pointer
	u32DataSize	Input data size.
Returns	N/A	
Referenced By	Crc_SetChannelCalculate (),Crc_CalculateCRC16(),Crc_CalculateCRC16ARC()	

2.7.1.3 uint32 CRC_GetCrcResult(uint8 u8Instance)

Function	uint32 CRC_GetCrcResult(uint8 u8Instance)
Description	Stop a hardware CRC channel

Diagram	 <pre> graph LR A[CRC_GetCrcResult] --> B[CRC_HWA_GetBitWidth] A --> C[CRC_HWA_GetData_U16_H] A --> D[CRC_HWA_GetData_U16_L] A --> E[CRC_HWA_GetData_U32] A --> F[CRC_HWA_GetReadDataSwap] </pre>	
Parameters	Parameter	Description
	u8Instance	CRC instance index.
Returns	CRC result calculated by hardware.	
Referenced By	Crc_SetChannelCalculate(), Crc_CalculateCRC16(), Crc_CalculateCRC16ARC()	

2.7.1.4 void CRC_SetSeed(uint8 u8Instance, uint32 u32SeedVal)

Function	void CRC_SetSeed(uint8 u8Instance, uint32 u32SeedVal)	
Description	Set CRC hardware channel pin state.	
Diagram	 <pre> graph LR A[CRC_SetSeed] --> B[CRC_HWA_SetData_U32] A --> C[CRC_HWA_SetDataOrSeed] </pre>	
Parameters	Parameter	Description
	u8Instance	CRC instance index.
	u32SeedVal	Seed value.
Returns	N/A	
Referenced By	Crc_SetChannelCalculate(), Crc_CalculateCRC16(), Crc_CalculateCRC16ARC(), Crc_CalculateCRC32(), Crc_CalculateCRC32P4(), Crc_Hw_Init()	

2.8 Software Functions

2.8.1 Functions in Crc_Lookup_Tables.h

2.8.1.1 uint8 Crc_TableCalculateCRC8(uint8 u8CrcLastResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, const Crc_ProtocolType eType)

Function	uint8 Crc_TableCalculateCRC8(uint8 u8CrcLastResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, const Crc_ProtocolType eType);	
Description	This function Calculate CRC8 Lookup Table with Channel Input: CRC8 and CRC8H2F	
Diagram	N/A	
Parameters	Parameter	Description
	u8CrcLastResult	Start value when the algorithm starts.
	Crc_DataPtr	Pointer to the Crc Data Input.
	Crc_Length	Length of crcDataPtr block to be calculated in bytes.

	eType	CRC8 and CRC8H2F
Returns	CRC final result: 8 bits	
Referenced By	Crc_CalculateCRC8(), Crc_CalculateCRC8H2F()	

2.8.1.2 uint16 Crc_TableCalculateCRC16(uint16 u16CrcLastResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, const Crc_ProtocolType eType)

Function	uint16 Crc_TableCalculateCRC16(uint16 u16CrcResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, const Crc_ProtocolType eType, const Crc_InitType *sProtocolCrc16)	
Description	This function Calculate CRC16 Lookup Table with Channel Input: CRC16 and CRC16ARC	
Diagram	N/A	
Parameters	Parameter	Description
	u16CrcLastResult	Start value when the algorithm starts.
	Crc_DataPtr	Pointer to the Crc Data Input.
	Crc_Length	Length of crcDataPtr block to be calculated in bytes.
	eType	CRC16 and CRC16ARC
Returns	CRC final result: 16 bits	
Referenced By	Crc_CalculateCRC16(), Crc_CalculateCRC16ARC()	

2.8.1.3 uint32 Crc_TableCalculateCRC32(uint32 u32CrcLastResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, const Crc_ProtocolType eType)

Function	uint32 Crc_TableCalculateCRC32(uint32 u32CrcResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, const Crc_ProtocolType eType, const Crc_InitType *sProtocolCrc32)	
Description	This function Calculate CRC32 Software with Channel Input: CRC32E2E_P4 and CRC32ETHERNET	
Diagram	N/A	
Parameters	Parameter	Description
	u32CrcLastResult	Start value when the algorithm starts.
	Crc_DataPtr	Pointer to the Crc Data Input.
	Crc_Length	Length of crcDataPtr block to be calculated in bytes.
	eType	CRC32E2E_P4 and CRC32ETHERNET.
Returns	CRC final result: 32 bits	
Referenced By	Crc_CalculateCRC32(), Crc_CalculateCRC32P4()	

2.8.1.4 uint64 Crc_TableCalculateCRC64(uint64 u64CrcLastResult, const uint8 *Crc_DataPtr, uint32 Crc_Length)

Function	uint64 Crc_TableCalculateCRC64(uint64 u64CrcResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, const Crc_InitType *sProtocolCrc64)	
Description	This function Calculate CRC64 Software with Channel Input: CRC64ECMA	
Diagram	N/A	
Parameters	Parameter	Description
	u64CrcLastResult	Start value when the algorithm starts.
	Crc_DataPtr	Pointer to the Crc Data Input.
	Crc_Length	Length of crcDataPtr block to be calculated in bytes.
Returns	CRC final result: 64 bits	

Referenced By	Crc_CalculateCRC64()
----------------------	----------------------

2.8.2 Functions in Crc_RunTime.h

2.8.2.1 uint8 Crc_RunTimeCalculateCRC8(uint8 u8CrcLastResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, uint8 u8Polynomial)

Function	uint8 Crc_RunTimeCalculateCRC8(uint8 u8CrcLastResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, uint8 u8Polynomial)	
Description	This function Calculate CRC8 Software with Channel Input: CRC8 and CRC8H2F	
Diagram	N/A	
Parameters	Parameter	Description
	u8CrcLastResult	Start value when the algorithm starts.
	Crc_DataPtr	Pointer to the Crc Data Input.
	Crc_Length	Length of crcDataPtr block to be calculated in bytes.
	u8Polynomial	The polynomial of CRC8 or CRC8H2F.
Returns	CRC final result: 8 bits	
Referenced By	Crc_CalculateCRC8(), Crc_CalculateCRC8H2F()	

2.8.2.2 uint16 Crc_RunTimeCalculateCRC16(uint16 u16CrcLastResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, const Crc_InitType * sProtocolCrc16)

Function	uint16 Crc_RunTimeCalculateCRC16(uint16 u16CrcLastResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, const Crc_InitType * sProtocolCrc16)	
Description	This function Calculate CRC16 Software with Channel Input: CRC16 and CRC16ARC	
Diagram	N/A	
Parameters	Parameter	Description
	u16CrcLastResult	Start value when the algorithm starts.
	Crc_DataPtr	Pointer to the Crc Data Input.
	Crc_Length	Length of crcDataPtr block to be calculated in bytes.
	sProtocolCrc16	CRC16 information: CRC16 and CRC16ARC.
Returns	CRC final result: 16 bits	
Referenced By	Crc_CalculateCRC16(), Crc_CalculateCRC16ARC()	

2.8.2.3 uint32 Crc_RunTimeCalculateCRC32(uint32 u32CrcLastResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, uint32 u32Polynomial)

Function	uint32 Crc_RunTimeCalculateCRC32(uint32 u32CrcLastResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, uint32 u32Polynomial)	
Description	This function Calculate CRC32 Software with Channel Input: CRC32E2E_P4 and CRC32ETHERNET	
Diagram	N/A	
Parameters	Parameter	Description
	u32CrcLastResult	Start value when the algorithm starts.
	Crc_DataPtr	Pointer to the Crc Data Input.
	Crc_Length	Length of crcDataPtr block to be calculated in bytes.

	u32Polynomial	The polynomial of CRC32E2E_P4 or CRC32ETHERNET.
Returns	CRC final result: 32 bits	
Referenced By	Crc_CalculateCRC32(), Crc_CalculateCRC32P4 ()	

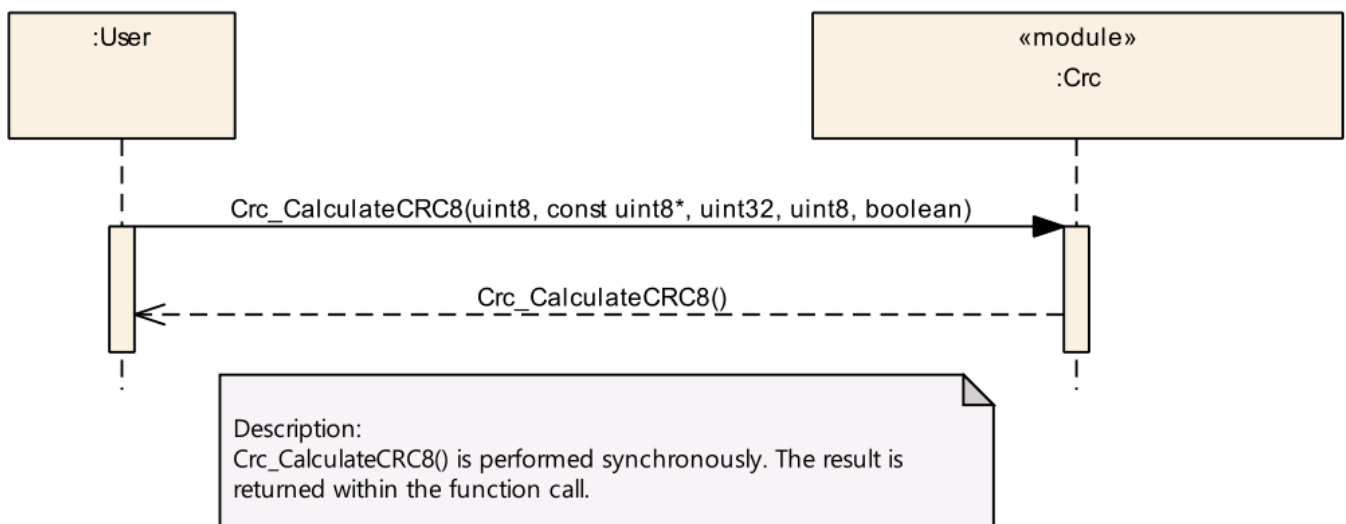
2.8.2.4 uint64 Crc_RunTimeCalculateCRC64(uint64 u64CrcLastResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, uint64 u64Polynomial)

Function	uint64 Crc_RunTimeCalculateCRC64(uint64 u64CrcLastResult, const uint8 *Crc_DataPtr, uint32 Crc_Length, uint64 u64Polynomial)	
Description	This function Calculate CRC64 Software with Channel Input: CRC64ECMA	
Diagram	N/A	
Parameters	Parameter	Description
	u64CrcLastResult	Start value when the algorithm starts.
	Crc_DataPtr	Pointer to the Crc Data Input.
	Crc_Length	Length of crcDataPtr block to be calculated in bytes.
	u64Polynomial	CRC64ECMA polynomial.
Returns	CRC final result: 64 bits	
Referenced By	Crc_CalculateCRC64()	

2.9 API Sequence Diagram

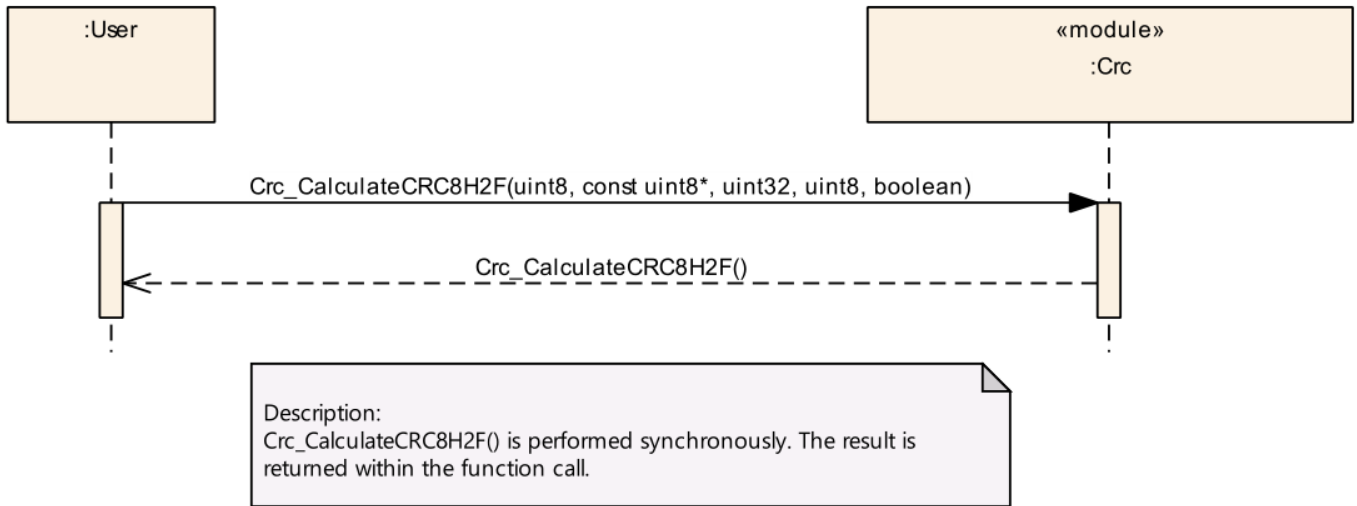
2.9.1 Crc_CalculateCRC8()

Crc_CalculateCRC8() is performed synchronously. The result is returned within the function call. The following diagram shows the synchronous function call Crc_CalculateCRC8.



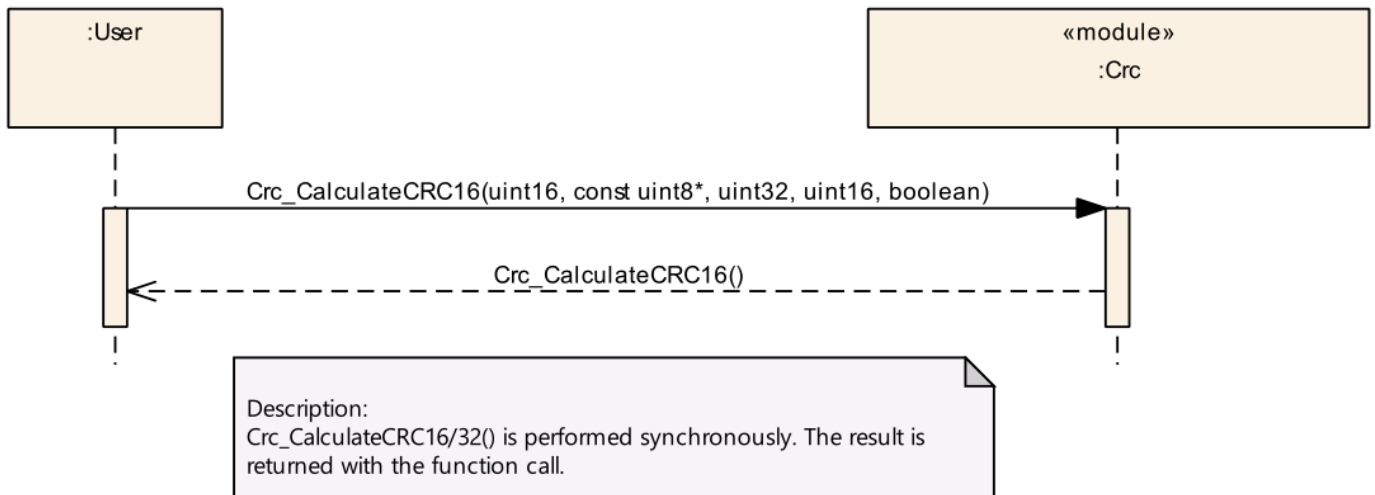
2.9.2 Crc_CalculateCRC8H2F()

Crc_CalculateCRC8H2F() is performed synchronously. The result is returned within the function call. The following diagram shows the synchronous function call Crc_CalculateCRC8H2F.



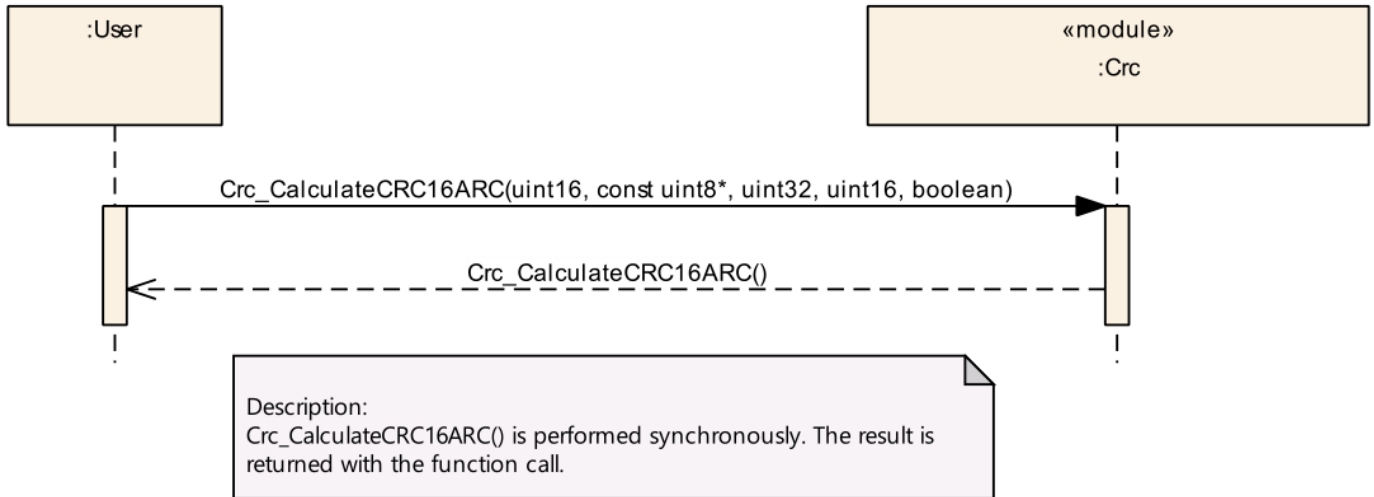
2.9.3 Crc_CalculateCRC16()

Crc_CalculateCRC16/32() is performed synchronously. The result is returned with the function call. The following diagram shows the synchronous function call Crc_CalculateCRC16.



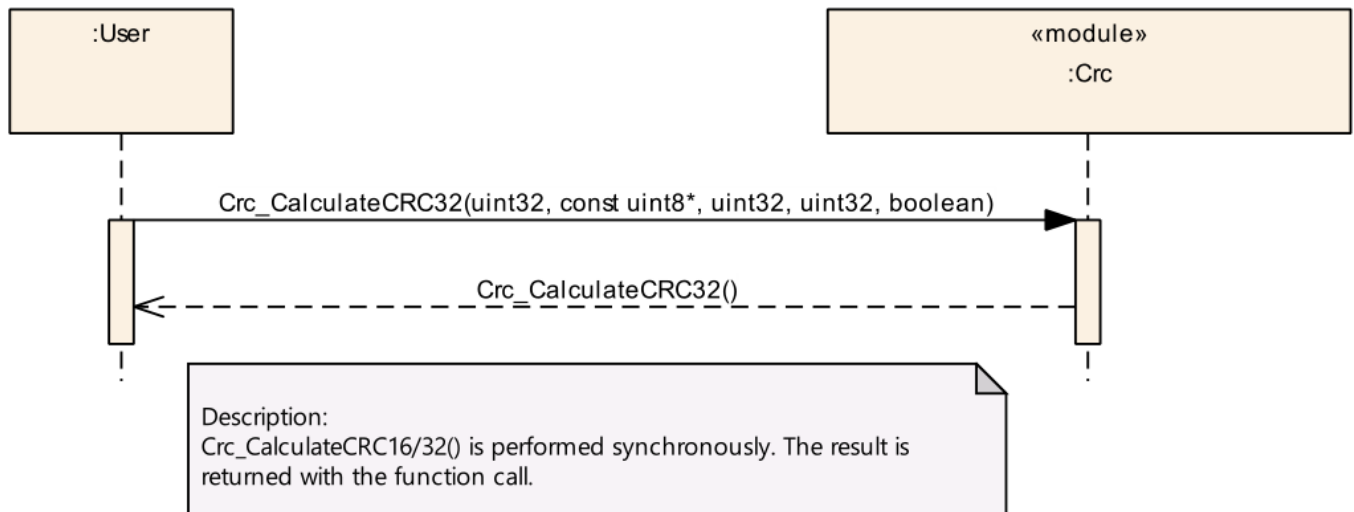
2.9.4 Crc_CalculateCRC16ARC()

Crc_CalculateCRC16ARC() is performed synchronously. The result is returned with the function call. The following diagram shows the synchronous function call Crc_CalculateCRC16ARC.



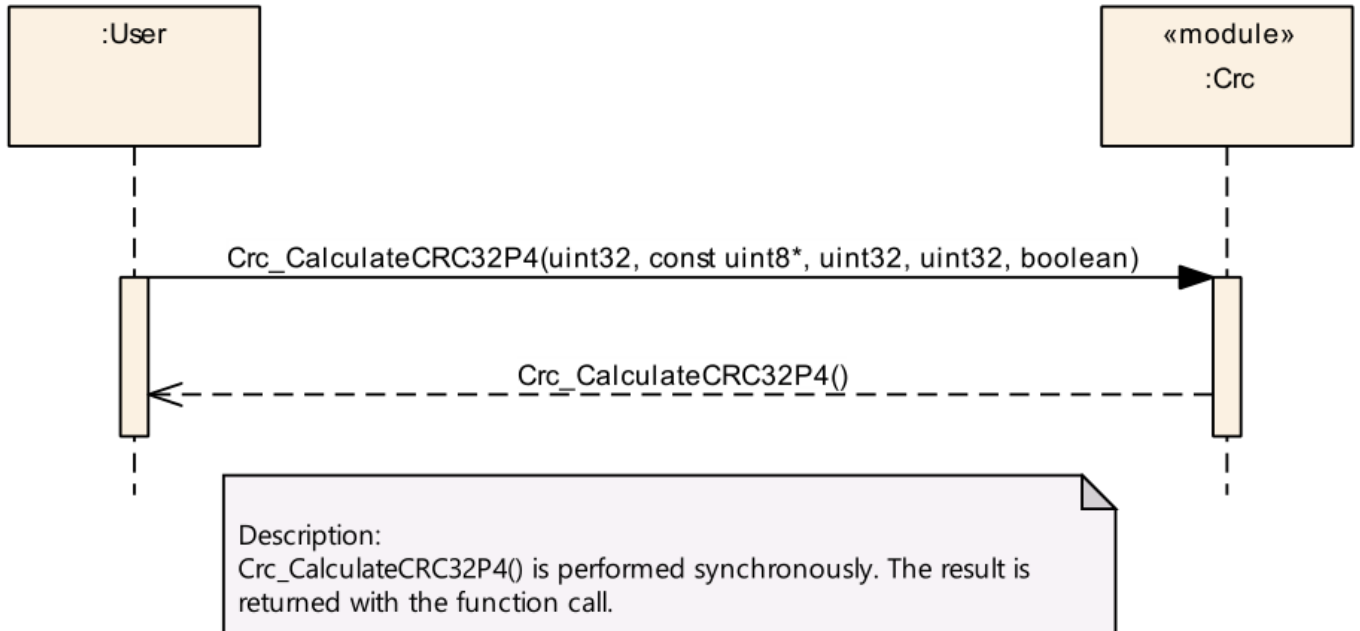
2.9.5 Crc_CalculateCRC32()

Crc_CalculateCRC16/32() is performed synchronously. The result is returned with the function call. The following diagram shows the synchronous function call Crc_CalculateCRC32.



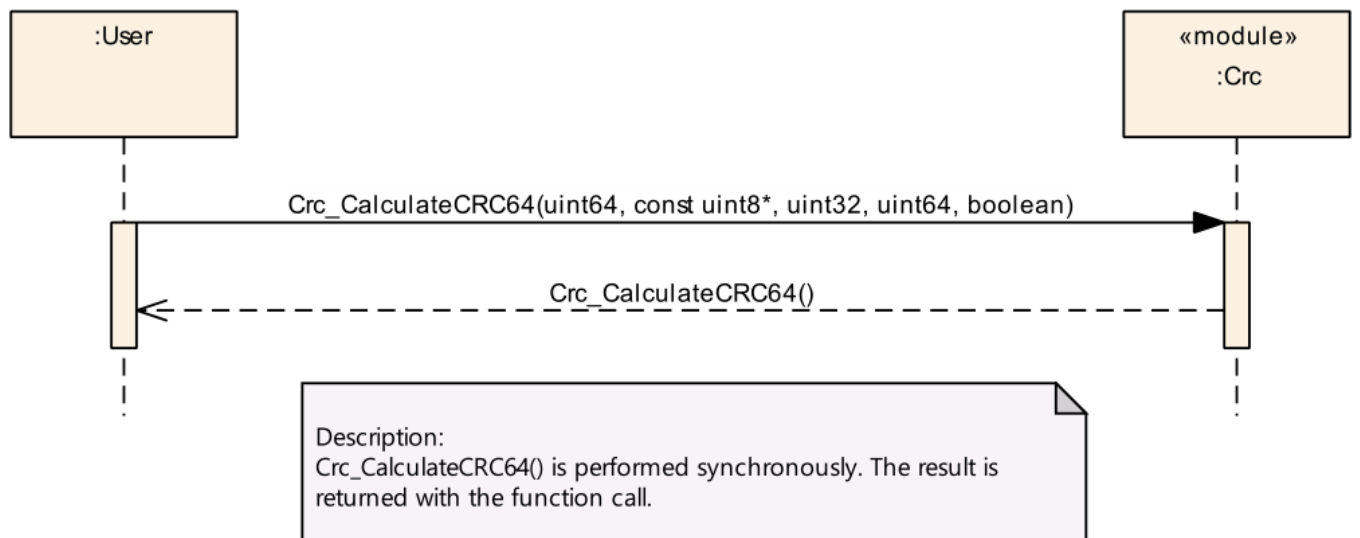
2.9.6 Crc_CalculateCRC32P4()

Crc_CalculateCRC32P4() is performed synchronously. The result is returned with the function call. The following diagram shows the synchronous function call Crc_CalculateCRC32P4.



2.9.7 Crc_CalculateCRC64()

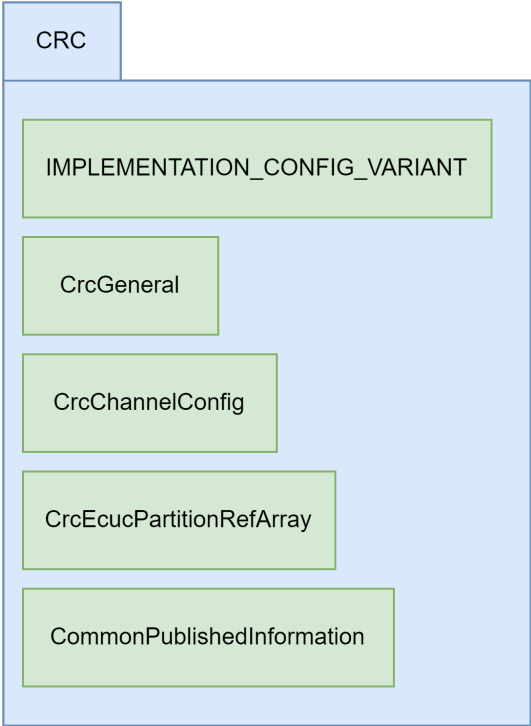
Crc_CalculateCRC64() is performed synchronously. The result is returned with the function call. The following diagram shows the synchronous function call Crc_CalculateCRC64.



Chapter 3 Tresos Configuration Items

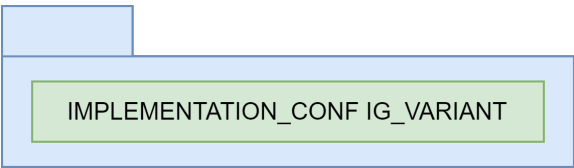
3.1 Container Inclusion Relation

The CRC module has the following 5 containers:

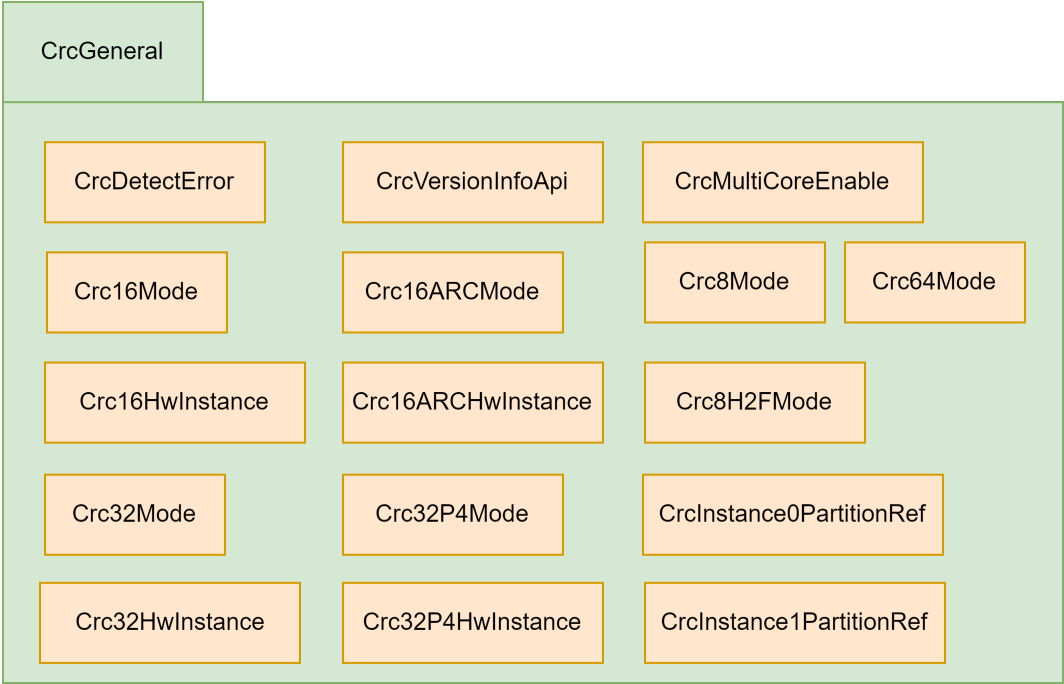


And the content of each container is as follows:

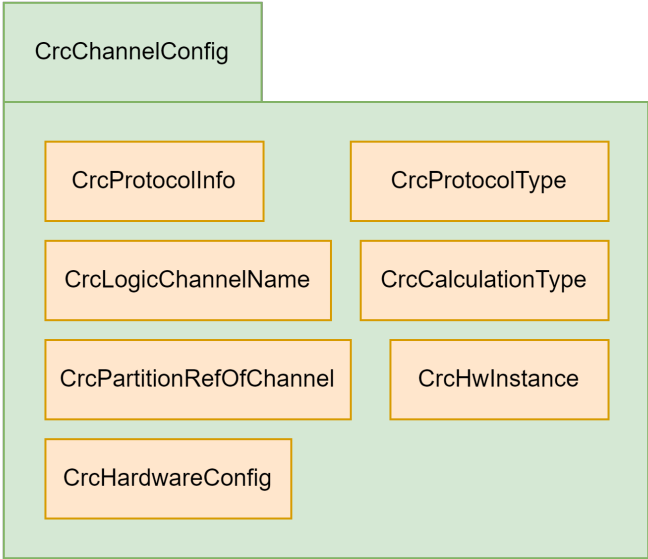
3.1.1 IMPLEMENTATION_CONFIG_VARIANT



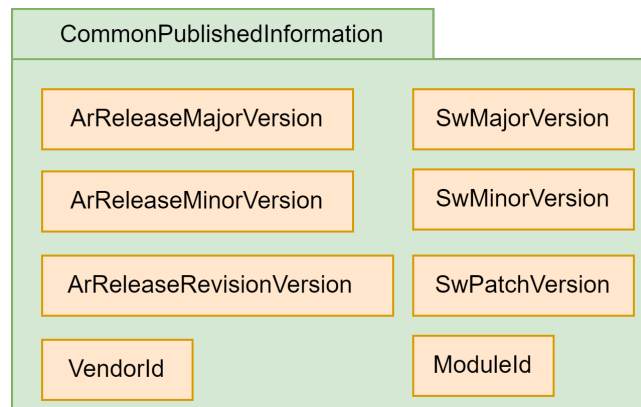
3.1.2 CrcGeneral



3.1.3 CrcChannelConfig



3.1.4 CommonPublishedInformation



3.2 Containers and Variables

3.3 IMPLEMENTATION_CONFIG_VARIANT

Container	IMPLEMENTATION_CONFIG_VARIANT	
Description	N/A	
Screenshot		
Properties	Property	Value
	Type	Variable: ENUMERATION
	Label	Config Variant
	Default	VariantPreCompile
	Range	VariantPreCompile
Returns	N/A	

3.3.1 CrcGeneral

Container	CrcGeneral	
Description	This container contains the module-wide configuration parameters of the CRC Driver.	
Screenshot		
Properties	Property	Value
	Type	Container

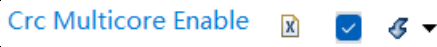
3.3.1.1 CrcDetectError

Variable	CrcDetectError	
Description	Switches the development error detection and notification on or off.	
Screenshot		
Properties	Property	Value
	Type	Variable: BOOLEAN
	Origin	AUTOSAR_ECUC
	SymbolicNameValue	false

3.3.1.2 CrcVersionInfoApi

Variable	CrcVersionInfoApi	
Description	Switch to indicate that the Crc_GetVersionInfo() is supported.	
Screenshot		
Properties	Property	Value
	Type	Variable: BOOLEAN
	Origin	AUTOSAR_ECUC
	SymbolicNameValue	false

3.3.1.3 CrcMultiCoreEnable

Variable	CrcMultiCoreEnable	
Description	Switches the multicore support on or off.	
Screenshot		
Properties	Property	Value
	Type	Variable: BOOLEAN
	Origin	FLAGCHIP
	SymbolicNameValue	false

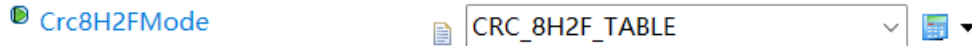
3.3.1.4 Crc8Mode

Variable	Crc8Mode	
Description	Switch to select one of the available CRC 8-bit (SAE J1850) calculation methods.	
Screenshot		
Properties	Property	Value
	Type	Variable: ENUMERATION
	Range	CRC_8_HARDWARE, CRC_8_RUNTIME, CRC_8_TABLE
	Origin	AUTOSAR_ECUC
	SymbolicNameValue	false

3.3.1.5 Crc8HwInstance

Variable	Crc8HwInstance	
Description	Identifies the Crc8 used Hardware Instance.	
Screenshot		
Properties	Property	Value
	Type	Variable: ENUMERATION
	Range	CRC_HW_INSTANCE_None, CRC_HW_INSTANCE_0
	Origin	FLAGCHIP
	SymbolicNameValue	false

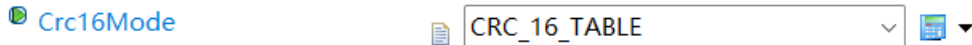
3.3.1.6 Crc8H2FMode

Variable	Crc8H2FMode	
Description	Switch to select one of the available CRC 8-bit (2Fh polynomial) calculation methods	
Screenshot		
Properties	Property	Value
	Type	Variable: ENUMERATION
	Range	CRC_8H2F_HARDWARE, CRC_8H2F_RUNTIME, CRC_8H2F_TABLE
	Origin	AUTOSAR_ECUC
	SymbolicNameValue	false

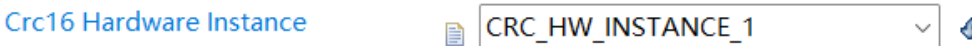
3.3.1.7 Crc8H2FHwInstance

Variable	Crc8H2FHwInstance	
Description	Identifies the Crc8H2F used Hardware Instance.	
Screenshot		
Properties	Property	Value
	Type	Variable: ENUMERATION
	Range	CRC_HW_INSTANCE_None, CRC_HW_INSTANCE_0
	Origin	FLAGCHIP
	SymbolicNameValue	false

3.3.1.8 Crc16Mode

Variable	Crc16Mode	
Description	Switch to select one of the available CRC 16-bit (CCITT) calculation methods	
Screenshot		
Properties	Property	Value
	Type	Variable: ENUMERATION
	Range	CRC_16_HARDWARE, CRC_16_RUNTIME, CRC_16_TABLE
	Origin	AUTOSAR_ECUC
	SymbolicNameValue	false

3.3.1.9 Crc16HwInstance

Variable	Crc16HwInstance	
Description	Identifies the Crc16 used Hardware Instance.	
Screenshot		
Properties	Property	Value
	Type	Variable: ENUMERATION
	Range	CRC_HW_INSTANCE_None, CRC_HW_INSTANCE_0, CRC_HW_INSTANCE_1
	Origin	FLAGCHIP

	SymbolicNameValue	false
--	-------------------	-------

3.3.1.10 Crc16ARCMODE

Variable	Crc16ARCMODE	
Description	Switch to select one of the available CRC-16/ARC (polynomial 8005) calculation methods	
Screenshot	 Crc16ARCMODE  CRC_16ARC_TABLE 	
Properties	Property	Value
	Type	Variable: ENUMERATION
	Range	CRC_16ARC_HARDWARE, CRC_16ARC_RUNTIME, CRC_16ARC_TABLE
	Origin	AUTOSAR_ECUC
	SymbolicNameValue	false

3.3.1.11 Crc16ARCHwInstance

Variable	Crc16ARCHwInstance	
Description	Identifies the Crc16ARC used Hardware Instance.	
Screenshot	 Crc16ARC Hardware Instance  CRC_HW_INSTANCE_0 	
Properties	Property	Value
	Type	Variable: ENUMERATION
	Range	CRC_HW_INSTANCE_None , CRC_HW_INSTANCE_0 , CRC_HW_INSTANCE_1
	Origin	FLAGCHIP
	SymbolicNameValue	false

3.3.1.12 Crc32MODE

Variable	Crc32MODE	
Description	Switch to select one of the available CRC 32-bit (IEEE-802.3 CRC32 Ethernet Standard) calculation methods	
Screenshot	 Crc32MODE  CRC_32_TABLE 	
Properties	Property	Value
	Type	Variable: ENUMERATION
	Range	CRC_32_HARDWARE, CRC_32_RUNTIME, CRC_32_TABLE
	Origin	AUTOSAR_ECUC
	SymbolicNameValue	false

3.3.1.13 Crc32HwInstance

Variable	Crc32HwInstance	
Description	Identifies the Crc32 used Hardware Instance.	
Screenshot	 Crc32 Hardware Instance  CRC_HW_INSTANCE_0 	
Properties	Property	Value
	Type	Variable: ENUMERATION
	Range	CRC_HW_INSTANCE_None , CRC_HW_INSTANCE_0 ,

		CRC_HW_INSTANCE_1
	Origin	FLAGCHIP
	SymbolicNameValue	false

3.3.1.14 Crc32P4Mode

Variable	Crc32P4Mode	
Description	Switch to select one of the available CRC 32-bit E2E Profile 4 calculation methods.	
Screenshot		
Properties	Property	Value
	Type	Variable: ENUMERATION
	Range	CRC_32P4_HARDWARE, CRC_32P4_RUNTIME, CRC_32P4_TABLE
	Origin	AUTOSAR_ECUC
	SymbolicNameValue	false

3.3.1.15 Crc32P4HwInstance

Variable	Crc32P4HwInstance	
Description	Identifies the Crc32P4 used Hardware Instance.	
Screenshot		
Properties	Property	Value
	Type	Variable: ENUMERATION
	Range	CRC_HW_INSTANCE_None , CRC_HW_INSTANCE_0 , CRC_HW_INSTANCE_1
	Origin	FLAGCHIP
	SymbolicNameValue	false

3.3.1.16 Crc64Mode

Variable	Crc64Mode	
Description	Switch to select one of the available CRC 64-bit calculation methods.	
Screenshot		
Properties	Property	Value
	Type	Variable: ENUMERATION
	Range	CRC_64_HARDWARE, CRC_64_RUNTIME, CRC_64_TABLE
	Origin	AUTOSAR_ECUC
	SymbolicNameValue	false

3.3.1.17 CrcInstance0PartitionRef

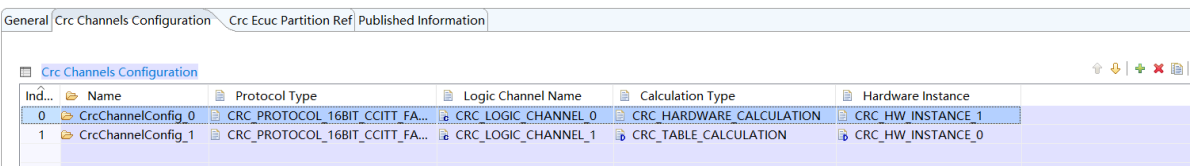
Variable	CrcInstance0PartitionRef	
Description	Maps the Crc instance0 driver to zero or one ECUC partitions to make the modules API available in this partition.	
Screenshot		

Properties	Property	Value
	Type	Variable: REFERENCE
	Origin	FLAGCHIP
	SymbolicNameValue	false

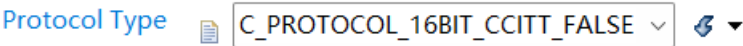
3.3.1.18 CrcInstance0PartitionRef

Variable	CrcInstance1PartitionRef	
Description	Maps the Crc instance1 driver to zero or one ECUC partitions to make the modules API available in this partition.	
Screenshot		
Properties	Property	Value
	Type	Variable: ENUMERATION
	Origin	FLAGCHIP
	SymbolicNameValue	false

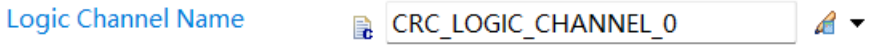
3.3.2 CrcChannelConfig

Variable	CrcChannelConfig	
Description	Configuration of an individual Crc channel. Symbolic names will be generated for each channel.	
Screenshot		
Properties	Property	Value
	Type	MAP
	Origin	FLAGCHIP
	SymbolicNameValue	false

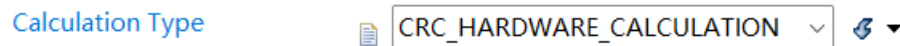
3.3.2.1 CrcProtocolType

Variable	CrcProtocolType	
Description	Identifies the Crc Protocol Type.	
Screenshot		
Properties	Property	Value
	Type	Variable: ENUMERATION
	Range	CRC_PROTOCOL_8BIT_SAE_J1850, CRC_PROTOCOL_8BIT_H2F, CRC_PROTOCOL_16BIT_CCITT_FALSE, CRC_PROTOCOL_16BIT_ARC, CRC_PROTOCOL_32BIT_ETHERNET, CRC_PROTOCOL_32BIT_E2E_P4, CRC_PROTOCOL_64BIT_ECMA
	Origin	FLAGCHIP
	Default	CRC_PROTOCOL_8BIT_SAE_J1850

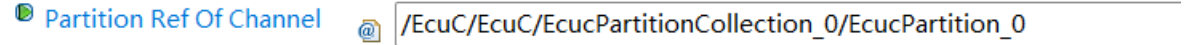
3.3.2.2 CrcLogicChannelName

Variable	CrcLogicChannelName	
Description	Channel used for Crc calculation	
Screenshot		
Properties	Property	Value
	Type	Variable: STRING
	Origin	FLAGCHIP

3.3.2.3 CrcCalculationType

Variable	CrcCalculationType	
Description	Select Crc Calculation Type.	
Screenshot		
Properties	Property	Value
	Type	Variable: ENUMERATION
	Origin	FLAGCHIP
	Range	CRC_TABLE_CALCULATION, CRC_HARDWARE_CALCULATION, CRC_RUNTIME_CALCULATION

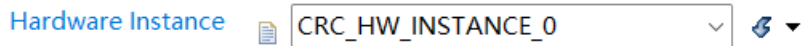
3.3.2.4 CrcPartitionRefOfChannel

Variable	CrcPartitionRefOfChannel	
Description	Maps a Crc hardware unit to zero or one ECUC partition to limit the access to this hardware unit. The ECUC partitions referenced are a subset of the ECUC partitions where the Crc hardware driver is mapped to.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

3.3.2.5 CrcHardwareConfig

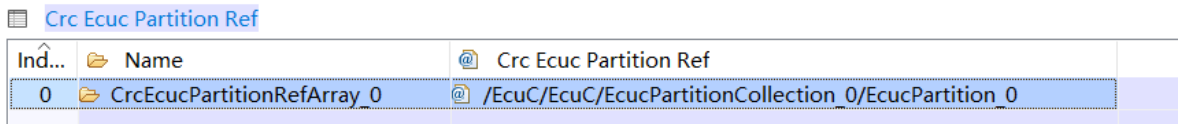
Variable	CrcHardwareConfig	
Description	This container contains the hardware configuration parameters of the Crc module.	
Screenshot		
Properties	Property	Value
	Type	Variable: IDENTIFIABLE
	Origin	FLAGCHIP

3.3.2.6 CrcHwInstance

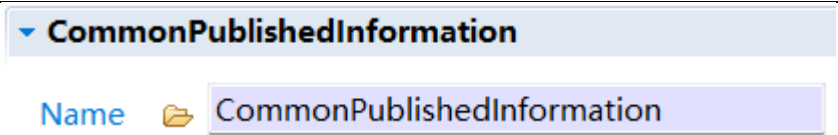
Variable	CrcHwInstance	
Description	Identifies the Crc Hardware Instance.	
Screenshot		
Properties	Property	Value

	Type	Variable: ENUMERATION
	Range	CRC_HW_INSTANCE_0, CRC_HW_INSTANCE_1
	Default	CRC_HW_INSTANCE_0
	Origin	FLAGCHIP

3.3.3 CrcEcucPartitionRef

Container	CrcEcucPartitionRef	
Description	Crc Ecuc Partition Ref	
Screenshot		
Properties	Property	Value
	Type	Container

3.3.4 CommonPublishedInformation

Container	CommonPublishedInformation	
Description	Common container, aggregated by all modules. It contains published information about vendor and versions.	
Screenshot		
Properties	Property	Value
	Type	Container

3.3.4.1 ArReleaseMajorVersion

Variable	ArReleaseMajorVersion	
Description	Major version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	Variable: INTEGER_LABEL
	Origin	FLAGCHIP
	SymbolicNameValue	false
	Default	4

3.3.4.2 ArReleaseMinorVersion

Variable	ArReleaseMinorVersion	
Description	Minor version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	Variable: INTEGER_LABEL
	Origin	FLAGCHIP

	SymbolicNameValue	false
	Default	6

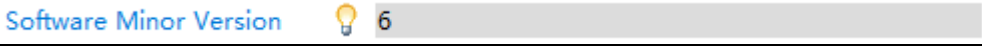
3.3.4.3 ArReleaseRevisionVersion

Variable	ArReleaseRevisionVersion	
Description	Revision version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot		
Properties	Property	Value
	Type	Variable: INTEGER_LABEL
	Origin	FLAGCHIP
	SymbolicNameValue	false
	Default	0

3.3.4.4 SwMajorVersion

Variable	SwMajorVersion	
Description	Major version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	Variable: INTEGER_LABEL
	Origin	FLAGCHIP
	SymbolicNameValue	false
	Default	0

3.3.4.5 SwMinorVersion

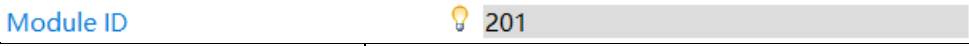
Variable	SwMinorVersion	
Description	Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	Variable: INTEGER_LABEL
	Origin	FLAGCHIP
	SymbolicNameValue	false
	Default	4

3.3.4.6 SwPatchVersion

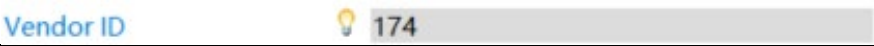
Variable	SwPatchVersion	
Description	Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value

	Type	Variable: INTEGER_LABEL
	Origin	FLAGCHIP
	SymbolicNameValue	false
	Default	0

3.3.4.7 ModuleId

Variable	ModuleId	
Description	Module ID of this module from Module List.	
Screenshot		
Properties	Property	Value
	Type	Variable: INTEGER_LABEL
	Origin	FLAGCHIP
	SymbolicNameValue	false
	Default	201

3.3.4.8 VendorId

Variable	VendorId	
Description	Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.	
Screenshot		
Properties	Property	Value
	Type	Variable: INTEGER_LABEL
	Origin	FLAGCHIP
	SymbolicNameValue	false
	Default	174

Chapter 4 Configuration Guides

4.1 Configuration Item Constraint

- 1) Hardware CRC do not support 8-bit (except FC7240) and 64-bit CRC. Hardware CRC only supports 16-bit and 32-bit CRC.

Crc Channels Configuration

▼ Crc Protocol Info

Name  CrcProtocolInfo

Protocol Type*  CRC_PROTOCOL_8BIT_SAE_J1850 

Logic Channel Name  CRC_LOGIC_CHANNEL_0 

Calculation Type  CRC_HARDWARE_CALCULATION 

Partition Ref Of Channel  /EcuC/EcuC/EcucPartitionCollection_0/EcucPartition_0

▼ CRC Hardware Config

Name  CrcHardwareConfig

Hardware Instance  CRC_HW_INSTANCE_0 

Crc Channels Configuration

▼ Crc Protocol Info

Name  CrcProtocolInfo

Protocol Type*  CRC_PROTOCOL_64BIT_ECMA 

Logic Channel Name  CRC_LOGIC_CHANNEL_0 

Calculation Type  CRC_HARDWARE_CALCULATION 

Partition Ref Of Channel  /EcuC/EcuC/EcucPartitionCollection_0/EcucPartition_0

▼ CRC Hardware Config

Name  CrcHardwareConfig

Hardware Instance  CRC_HW_INSTANCE_0 

- 2) Users can configure the CRC calculation type (table, runtime, hardware) in general Tab to control functions (Crc_CalculateCRC8 , Crc_CalculateCRC8H2F , Crc_CalculateCRC16 , Crc_CalculateCRC16ARC , Crc_CalculateCRC32, Crc_CalculateCRC32P4, Crc_CalculateCRC64) calculation type. If choosing hardware calculation type, user can distribute CRC hardware instance and assign instance to specify core when the multicore is enabled. This configuration can only configure one calculation type (runtime or table lookup or hardware) for the same CRC mode.
- 3) Users can configure multiple channels in the channelconfig tab to implement different CRC calculations. Compared with the second limitation, this method can configure different calculation methods for the same CRC mode.

4.2 CRC Usage Common Steps

Basically, the CRC module can be configured by the following steps:

- 1) Add a channel object in the Crc Channels Configuration tab.

General Crc Channels Configuration Crc Ecuc Partition Ref Published Information					
Crc Channels Configuration					
Ind...	Name	Protocol Type	Logic Channel Name	Calculation Type	Hardware Instance
0	CrcChannelConfig_0	CRC_PROTOCOL_16BIT_CCITT_FA...	CRC_LOGIC_CHANNEL_0	CRC_HARDWARE_CALCULATION	CRC_HW_INSTANCE_0
1	CrcChannelConfig_1	CRC_PROTOCOL_16BIT_CCITT_FA...	CRC_LOGIC_CHANNEL_1	CRC_TABLE_CALCULATION	CRC_HW_INSTANCE_0

2) Configure CRC protocol type and calculation type in Crc Channels Configuration tab.

Crc Channels Configuration

▼ Crc Protocol Info

Name

Protocol Type* 

Logic Channel Name 

Calculation Type* 

Partition Ref Of Channel

▼ CRC Hardware Config

Name

Hardware Instance 

3) Enable multicore and allocate EcucPartitions for each channel (Skip this step if multicore is disable)

General Crc Channels Configuration Crc Ecuc Partition Ref Published Information

Post Build Variant Used  

Config Variant

▼ Crc General

Name

Crc Development Error Detection  ☒  Crc Multicore Enable  ☒ 

General Crc Channels Configuration Crc Ecuc Partition Ref Published Information		
Crc Ecuc Partition Ref		
Ind...	Name	Crc Ecuc Partition Ref
0	CrcEcucPartitionRefArray_0	/EcuC/EcuC/EcucPartitionCollection_0/EcucPartition_0

Crc Channels Configuration

Crc Protocol Info

Name

CrcProtocolInfo

Protocol Type*

C_PROTOCOL_16BIT_CCITT_FALSE

Logic Channel Name

CRC_LOGIC_CHANNEL_0

Calculation Type*

CRC_HARDWARE_CALCULATION

Partition Ref Of Channel

/EcuC/EcuC/EcucPartitionCollection_0/EcucPartition_0

CRC Hardware Config

4) Generate configuration files.